

FIN-GUARDIAN AI

Autonomous Fraud Detection & Real-Time Mitigation

Full System Architecture & Design Document

From Zero to Understanding — A Complete Technical Narrative

Version 1.0 • 2026

0. The Story That Started Everything

Before we talk about technology, databases, or machine learning models, let us understand *why* this system exists — through a story that is completely real in spirit.

Story

It is 11:45 PM on a Tuesday night. Priya is a 21-year-old college student in Pune. She has a basic savings account. She has never sent more than ₹5,000 in a single transaction in her entire life. Somewhere in Mumbai, a criminal has stolen Priya's banking credentials using a phishing link she clicked three days ago. He opens her mobile app. He adds a brand-new recipient — someone Priya has never sent money to before. Then he initiates a transfer of ₹92,000. The old banking system sees this and thinks: "Is her account balance sufficient? Yes. Is the PIN correct? Yes. Approved." By the time Priya wakes up the next morning, ₹92,000 is gone. She calls the bank. The money has already been moved three times across four accounts in different states and is virtually untraceable. This story happens thousands of times every day across India and the world.

This is the problem Fin-Guardian AI was built to solve. Not to simply send Priya an SMS saying "suspicious activity detected." By that point, it is too late. Fin-Guardian AI is built to catch the criminal *while his finger is still on the screen* — in under 30 milliseconds — before the money moves at all.

1. What Is Wrong with Today's Systems?

To appreciate the solution, we must first understand exactly why existing fraud systems fail. There are three core problems:

Problem 1 — They React, They Do Not Prevent

Traditional fraud detection works like a security guard who reviews CCTV footage the morning after a break-in. The system runs batch jobs — perhaps every hour, or even once a day — scanning past transactions for red flags. By the time it identifies fraud, the money is long gone. This is called a reactive system.

The Core Flaw

A system that detects fraud 12 hours after it happens is not a fraud prevention system. It is a fraud documentation system. It creates a report. The victim is already harmed.

Problem 2 — They Are Brittle and Binary

Most rule-based systems work on simple if-then logic, for example: "If the transaction amount is above ₹50,000, block it." This creates two catastrophic failure modes:

- Legitimate customers are wrongly blocked — a salaried professional sending rent to their landlord gets denied. This destroys trust.
- Fraudsters learn the exact rules — they simply send ₹49,999 instead of ₹50,000 and the system lets them through. This is called adversarial adaptation.

Problem 3 — They See Each Transaction in Isolation

A single transaction of ₹92,000 from Priya's account might look suspicious. But what if you could see that the recipient account received money from 47 different victims in the past month? What if that recipient immediately forwarded the money to two other accounts, which then immediately forwarded it again? This is a money laundering network — and traditional systems completely miss it because they only look at one transaction at a time, never the web of connections between accounts.

2. The Big Idea Behind Fin-Guardian AI

Fin-Guardian AI is built on one fundamental insight:

Fraud is not a single data point. It is a pattern across time, behaviour, networks, and context.

Rather than asking "Is this transaction large?", Fin-Guardian AI asks five simultaneous questions:

- Is this transaction statistically normal for THIS specific user, at THIS time of day, to THIS type of recipient?
- Does the recipient account belong to a known criminal network when we map all financial relationships as a graph?
- Is the device being used showing signs of remote control, spoofing, or automation?
- Are there missing contextual signals — like no GPS data at midnight — that suggest the real user is not present?
- What is the minimum disruption we can apply to verify this is legitimate, without destroying the experience for genuine customers?

The Philosophy

Fin-Guardian AI does not try to block fraud. It tries to understand context deeply enough that it knows the difference between Priya choosing to send money at midnight and a criminal using Priya's account at midnight. Only one of those should be stopped.

3. How It Works — The Four Defence Layers

Think of Fin-Guardian AI as a building with four security checkpoints. Every transaction passes through each checkpoint, but the checkpoints work at very different speeds and depths. The fastest ones make an immediate decision. The slower ones provide deep intelligence in the background.

Story

Back to Priya's story. The criminal presses "Send" on the ₹92,000 transfer. That single button press triggers all four layers simultaneously. Here is what happens in the next few seconds inside Fin-Guardian AI...

Layer 1 — The Instant Check (Hot Path, under 30 milliseconds)

The moment the transaction request arrives, Fin-Guardian AI has **30 milliseconds** to make a decision. For context, a human eye-blink takes 150-400 milliseconds. This system must think, evaluate, and respond in one-fifth of that time.

Two specialised AI engines fire simultaneously:

1A

The Fraud Classifier (XGBoost Model)

Trained on millions of historical fraud cases, this model scores the transaction against dozens of features: Is ₹92,000 unusual for this account? Is 11:45 PM an unusual time for this user? Is this a first-time recipient? Is the account a low-velocity student profile? It produces a fraud probability score in milliseconds.

1B

The Behaviour Engine (Autoencoder Neural Network)

This AI has learned what 'normal Priya' looks like — her typical transaction sizes, times, patterns. It compares the current transaction against her personal profile and measures how far it deviates. A high deviation score means 'this does not feel like Priya.' This catches novel frauds that the classifier has never seen before.

1C

The Explainability Engine (SHAP)

Crucially, the system does not just say yes or no. It produces a human-readable reason: 'Transaction flagged because: amount is 18x the account average, recipient is new (0 previous transactions), time is 3 standard deviations from user's normal activity window.' This is legally required for regulatory compliance.

Based on these three engines, Layer 1 issues one of three verdicts:

Verdict	Meaning	What Happens Next
APPROVE	High confidence this is legitimate	Transaction goes through instantly. Customer notices nothing.
HOLD	Suspicious — needs deeper investigation	Transaction is paused. Deep investigation starts immediately in background.
DENY	High confidence this is fraud	Transaction is blocked. Customer is notified with explanation.

In our story, Layer 1 issues a **HOLD** verdict. The criminal's transfer does not go through. But the system does not simply throw up an error. It immediately signals deeper layers to begin their investigation — *while the criminal is still waiting for a confirmation screen.*

Layer 2 — The Network Map (Warm Path, seconds to minutes)

The moment a HOLD is issued, Layer 2 begins running in the background. It asks a completely different type of question: "Who is this recipient, really — in the context of ALL financial relationships in our system?"

📖 Story

The recipient account the criminal chose received ₹7,000 from someone in Delhi yesterday. And ₹15,000 from someone in Kolkata last week. And ₹34,000 from someone in Hyderabad three days ago. None of these senders know each other. But this single recipient connects them all. This is the fingerprint of a money mule account — a criminal collector point. No single transaction reveals this. Only the network map does.

Layer 2 uses Neo4j, a graph database, to model every account as a node and every transaction as an edge (a connection between nodes). Then it runs sophisticated mathematical algorithms:

- **Community Detection (Louvain Algorithm):** Identifies tightly-connected clusters of accounts that are transacting with each other unusually frequently — the signature of a money laundering ring.
- **Centrality Analysis:** Identifies accounts that are central connectors between many other accounts — the money mules and collection points.

- **Relationship Mapping:** Checks if the recipient is connected to any previously identified fraud cases, even through two or three degrees of separation.

Why This Matters

A sophisticated criminal never uses one account. They build networks of 50-100 accounts that pass money between each other to make it untraceable. Graph analysis sees the entire web, not just one thread.

Layer 3 — The AI Detective (Deep Path, the Agentic Investigation)

This is the most innovative and powerful layer. It is an AI agent — a system that does not just calculate a score but actively investigates, like a detective following leads. It is built using LangGraph, an AI orchestration framework.

📖 Story

In Priya's case, the system noticed something alarming: there is no GPS data attached to this transaction. Priya's phone has location services enabled. Every previous transaction she has made included her location. This one has none. This is a red flag — it suggests the transaction is being made by software, not by a person physically holding the phone. The AI Detective is activated to investigate this missing signal.

The AI Detective has access to a set of specialised investigation tools it can call in sequence, like a detective calling different specialists:

Tool 1

IP Geolocation Lookup

The transaction came from IP address X.X.X.X. Where in the world is that IP address? Is it in Pune where Priya lives? Is it masked by a VPN? Is it a known Tor exit node used by criminals? Is it in a different country entirely?

Tool 2

Device Fingerprint Extractor

What device is being used? Is it Priya's known Samsung Galaxy, or a new unknown device? Is the operating system real or emulated (emulators are used by criminals to run many fake sessions)? Are there signs of a remote desktop connection? Is a bot framework present?

Tool 3

Graph Database Query

The detective can write and execute queries directly against the Neo4j graph database in real time: "Find all accounts that have received money from the suspect recipient in the past 30 days and check if any are flagged as fraud." This is done with precise, controlled queries — not guesswork.

Tool
4

Risk Matrix Fallback

If any tool fails or takes too long (more than 2 seconds), the agent automatically falls back to a pre-defined risk matrix. The investigation never hangs or crashes — it degrades gracefully.

The AI agent synthesises all findings from its tools and produces a **forensic investigation report** — a structured document explaining exactly what it found, why it is suspicious, and what it recommends. This report feeds into Layer 4.

Layer 4 — The Smart Response (Mitigation, not Destruction)

The old approach to fraud mitigation was simple and brutal: block the transaction and lock the account. Fin-Guardian AI takes a fundamentally different philosophy:

Old System — Hard Block Block the transaction for everyone, always. <i>Result: Criminals are blocked. But so are innocent customers. High false positive rate destroys user trust.</i>	Fin-Guardian AI — Step-Up Challenge Ask the user to prove they are human — only when suspicious. <i>Result: Criminals cannot complete the challenge. Legitimate users pass easily. Experience preserved.</i>
--	--

The step-up challenges are adaptive — they escalate based on the risk score:

- Low suspicion: A simple one-tap confirmation — "Did you initiate this transfer?"
- Medium suspicion: A biometric re-authentication — the user must use Face ID or fingerprint again.
- High suspicion: A behavioural challenge — the user must swipe or interact in a way that proves human presence. Bots cannot replicate natural human touch patterns.
- Confirmed fraud: The transaction is blocked, the account is flagged, and the security team is alerted with the full investigation report.

Story

Back to the criminal. He pressed Send on Priya's ₹92,000 transfer. The confirmation screen never came. Instead, a biometric challenge appeared: 'Please verify your identity using Face ID to continue.' The criminal has Priya's password. He does not have her face. He stares at the screen for a few seconds, then closes the app. The transfer never happens. Priya wakes up the next morning, opens her banking app, and sees a notification: 'Suspicious login detected. We protected your account. Tap here to review.' Her ₹92,000 is safe. This entire sequence — from the criminal pressing Send to the challenge appearing — took 847 milliseconds.

4. The Technology Stack — Every Tool and Why

Every technology chosen in Fin-Guardian AI was selected to solve a specific engineering constraint. Here is the complete stack with plain-language explanations:

Component	Technology	Why This Choice
Ingestion Gateway	FastAPI (Python)	The fastest Python web framework. Built for async operations — it can handle thousands of simultaneous requests without waiting for each to complete before starting the next.
Data Validation	Pydantic V2	Every transaction must be perfectly structured before processing. Pydantic enforces this automatically and rejects malformed data at the door.
Fraud Classifier	XGBoost + ONNX Runtime	XGBoost is industry-standard for tabular fraud data. ONNX Runtime accelerates inference to sub-5ms — critical for the 30ms SLA.
Behaviour Engine	PyTorch Autoencoder	Neural networks that learn normal user behaviour patterns. Detects novel fraud that rule-based systems miss entirely.
Explainability	SHAP	Produces human-readable reasons for every decision. Required by financial regulators — a black-box AI is not legally acceptable for blocking transactions.
Event Bus	Apache Kafka	A durable, replayable message queue. When a HOLD is issued, the event is written to Kafka. Workers pick it up. If a worker crashes, the event is not lost — it replays automatically.
Async Workers	Celery + Arq	Background task runners. Graph queries, AI agent investigations, and device fingerprinting all run here — completely decoupled from the hot path.
Graph Database	Neo4j + GDS	The only database built specifically for relationship queries. Finding a money

Component	Technology	Why This Choice
		laundering network across 10,000 accounts takes milliseconds in Neo4j. It would take minutes in a traditional SQL database.
AI Agent	LangGraph + Gemini/GPT-4	A state machine that orchestrates the AI detective investigation. The LLM decides which tools to use and synthesises findings. All actual data access is through hard-coded, deterministic Python functions — not AI guesswork.
Feature Store	Redis + Feast	Pre-computed user features (rolling averages, velocity scores) stored in memory. Hot-path inference reads these in ~1ms instead of computing them live from the database.
Dashboard	HTML5 + WebSockets	A live command centre for the security team. Alerts appear in real time as transactions are flagged. No page refresh needed — WebSockets push updates instantly.
Observability	Loguru + OpenTelemetry	Structured JSON logging for every system event. End-to-end distributed tracing so engineers can see exactly what happened in any 847-millisecond fraud detection sequence.

5. The Core Engineering Rules

Fin-Guardian AI is governed by strict engineering principles. These are not preferences — they are hard constraints that ensure the system is reliable, fair, and legally defensible.

Rule 1 — The 30-Millisecond Law

The critical path — from receiving a transaction to returning APPROVE, HOLD, or DENY — must complete in under 30 milliseconds. This is non-negotiable. To achieve this, only two things are allowed on the critical path: the XGBoost classifier and the Autoencoder. Everything else — graph queries, AI agents, device fingerprinting — runs asynchronously in the background and reports findings after the initial decision.

Rule 2 — Never Block the Thread

When a HOLD is issued, the system must immediately release the client connection and dispatch the investigation event to Kafka. The client thread is not held open while the investigation runs. This is why the system can handle thousands of simultaneous fraud investigations without slowing down.

Rule 3 — The AI Is a Router, Not an Oracle

The LLM (Large Language Model) inside the AI detective is strictly not allowed to look up data, make calculations, or access any external system directly. Its only jobs are: deciding which investigation tool to use next, and synthesising the results into a coherent report. All actual data retrieval is done by hard-coded Python functions. This prevents hallucination, ensures reproducibility, and maintains an auditable investigation trail.

Rule 4 — Graceful Degradation

If Neo4j goes offline, the system does not crash. The circuit breaker activates and the investigation falls back to a pre-defined rule matrix. If the LLM API times out, the agent uses its last available findings. If the feature store is unavailable, features are computed live from the database (slower, but functional). The system is designed to degrade gracefully, never to fail catastrophically.

Rule 5 — Every Decision Is Explainable

A transaction cannot be blocked without a human-readable reason. The SHAP engine produces this for every decision. This is both an ethical requirement — customers deserve to know why they were challenged — and a legal requirement under financial regulations in India, Europe, and globally.

6. Known Challenges and Engineering Solutions

No system design is perfect on paper. Here are the critical gaps identified in Fin-Guardian AI and how they are resolved:

Challenge 1 — Time Feature Encoding

The fraud classifier uses the hour of a transaction as a feature. But raw hour numbers are misleading to a machine learning model. Hour 23 (11 PM) and hour 0 (midnight) are only 60 minutes apart, but numerically they look like they are 23 hours apart. This causes the model to misread temporal proximity.

Solution — Cyclical Encoding

Time features are encoded as sine and cosine values: $\sin(2\pi \times \text{hour} / 24)$ and $\cos(2\pi \times \text{hour} / 24)$. This maps time onto a circle, so 11:45 PM and 12:15 AM are correctly understood as 30 minutes apart. The same technique is applied to day-of-week features.

Challenge 2 — Kafka Message Loss on Worker Crash

If a Celery worker crashes mid-investigation for a HOLD transaction, the investigation is lost. Redis Pub/Sub (the simpler option) has zero message durability — a crash means permanent data loss.

Solution — Kafka with Consumer Groups and DLQ

All HOLD events use Apache Kafka with consumer groups. Kafka retains every message for 7 days. If a worker crashes, another worker picks up the message automatically. Events that fail after 3 attempts are routed to a Dead Letter Queue (DLQ) with alerting.

Challenge 3 — Graph Database Becoming Stale

The Neo4j fraud graph is only useful if it is current. A graph that does not know about transactions from the past hour will miss newly created money mule accounts.

Solution — Async Graph Writer

Every resolved transaction (both approved and denied) triggers a background `graph_writer` task that writes new edges to Neo4j within seconds. The graph is always a live representation of the financial network. Old nodes are archived after 90 days to maintain performance.

Challenge 4 — Hot Path Latency on Feature Computation

Computing features like '30-day rolling average transaction amount' live from the database would require a complex SQL query for every single transaction. This alone could take 50-200 milliseconds — more than the entire 30ms budget.

Solution — Pre-Materialised Feature Store

A Redis-backed feature store pre-computes all per-user features on a rolling basis. The hot path reads a single `HGETALL` command (~1ms) to get all features instantly. Features are updated in the background by a streaming aggregation pipeline.

Challenge 5 — Biometric Data Privacy

Layer 4 may collect biometric challenge responses (facial geometry, touch patterns). Under India's DPDP Act 2023, biometric data requires explicit consent, must be minimised, and must have a documented retention and deletion policy.

Solution — Consent Model and Hashed Storage

A BiometricConsentRecord schema tracks explicit user consent timestamps. Biometric challenge responses are one-way hashed before storage — raw biometric data is never persisted. A scheduled purge pipeline deletes records older than the retention period automatically.

7. The Complete End-to-End Flow — One Transaction, Step by Step

Here is the complete journey of a single suspicious transaction through every layer of Fin-Guardian AI. Each step includes what is happening technically and what it means in plain English.

Step 1

Transaction Arrives at the Gateway

A mobile app sends a transaction request to FastAPI. Pydantic V2 validates the schema — checking all required fields, data types, and formats. If the request is malformed, it is rejected in under 1ms with a clear error message. If valid, it proceeds.

Step 2

Features Are Retrieved from the Feature Store

The gateway makes a single Redis call (HGETALL) to retrieve this user's pre-computed profile: average transaction amount, typical transaction times, velocity scores, recipient history. This takes approximately 1ms.

Step 3

XGBoost + Autoencoder Fire in Parallel

Both models receive the transaction features simultaneously. XGBoost scores the transaction against historical fraud patterns (~3ms). The Autoencoder measures deviation from this user's personal baseline (~4ms). Total: approximately 5ms.

Step 4

SHAP Generates the Explanation

SHAP calculates which features contributed most to the fraud score and by how much. It produces a structured explanation: the top 3 reasons why this transaction is suspicious. (~3ms)

Step 5

Decision Is Made — HOLD Issued

The combined score exceeds the HOLD threshold. Total elapsed time: approximately 12ms. A HOLD verdict is created with the SHAP explanation attached.

Step 6

HOLD Event Dispatched to Kafka — Client Thread Released

The HOLD event payload is published to the Kafka topic 'transactions.hold'. The client receives an immediate response: 'Verification required.' The FastAPI thread is now free to handle other requests. Total response time: under 20ms.

Step 7

Neo4j Graph Analysis Begins (Async)

A Celery worker picks up the Kafka event and queries Neo4j. It runs Louvain community detection and centrality analysis on the recipient account. Finds: recipient is connected to 12 other accounts that have been flagged in the past 30 days. Risk score elevated.

Step 8

AI Detective Investigation Begins (Async)

A second Celery worker spins up the LangGraph agent. It calls the IP Geolocation tool: IP is from a VPN provider. It calls the Device Fingerprint tool: OS is emulated (not a real phone). It queries Neo4j: recipient account is a known money mule. Agent synthesises findings.

Step 9

Forensic Report Generated

The AI agent produces a complete forensic report: 'High confidence fraud. Evidence: VPN usage, emulated device, recipient is a tier-1 money mule connected to 12 fraud victims. Recommendation: Block transaction, flag account, alert security team.'

Step 10

Step-Up Challenge Delivered

Based on the investigation findings, the highest-level biometric challenge is triggered. The user sees: 'Please verify with Face ID.' The criminal cannot proceed. If a legitimate user triggers a HOLD (rare false positive), they complete the challenge in 3 seconds and proceed normally.

Step 11

Dashboard Alert Pushed

A WebSocket message is pushed to the security team dashboard in real time. The analyst sees the complete investigation report, the SHAP explanation, the graph visualisation, and the forensic findings. They can review and take manual action if needed.

8. What Success Looks Like

A system is only as good as its measurable outcomes. Here are the metrics that define success for Fin-Guardian AI:

Metric	Target	What It Means
Hot Path Latency (P99)	< 30ms	99% of all transactions receive a verdict in under 30 milliseconds.
False Positive Rate	< 0.5%	Fewer than 5 legitimate transactions in 1,000 are incorrectly challenged.
False Negative Rate	< 2%	Fewer than 2 fraud attempts in 100 slip through undetected.
System Uptime	99.9%	The ingestion gateway is available at least 99.9% of the time.

Metric	Target	What It Means
Graph Query Latency	< 500ms	Neo4j community detection completes within half a second.
Agent Investigation Time	< 30 seconds	The full AI detective investigation completes in under 30 seconds.
Feature Store Hit Rate	99%	99% of hot path requests find pre-computed features in Redis.
Audit Trail Completeness	100%	Every single decision — approve, hold, or deny — has a stored SHAP explanation.

9. Glossary — Plain Language Definitions

Every technical term used in this document, explained simply:

Term	Plain English Explanation
Async / Asynchronous	A task that starts and runs in the background without making everything else wait for it to finish.
Autoencoder	A type of AI that learns what 'normal' looks like, then measures how different something new is from normal.
Circuit Breaker	A safety switch. If a downstream service fails too many times, the circuit breaker trips and the system uses a safe fallback instead of keep trying and crashing.
Cypher	The query language used to talk to the Neo4j graph database. Like SQL, but for graphs and relationships.
Feature Store	A fast database (Redis) that stores pre-computed user statistics so the hot path can retrieve them in 1ms without calculating them live.
Graph Database	A database designed to store and query relationships between things, not just things themselves.
Hot Path	The critical, time-sensitive route a transaction takes that must complete in under 30ms.
Kafka	A message bus system where producers write events and consumers read them. Events are stored durably, so nothing is lost if a consumer crashes.

Term	Plain English Explanation
LangGraph	A framework for building AI agents that follow a defined state machine — so the AI behaves in predictable, auditable steps.
Louvain Algorithm	A mathematical method for finding tightly-connected groups (communities) within a large network.
ONNX Runtime	A tool that accelerates AI model inference. It converts models into an optimised format that runs 3-10x faster than the original.
Pydantic	A Python library that validates data structures. If a transaction is missing a required field or has a wrong data type, Pydantic catches it instantly.
Reconstruction Error	How badly an autoencoder fails to recreate a transaction from its compressed representation. High error = very abnormal transaction.
SHAP	A mathematical method (SHapley Additive exPlanations) for explaining why a machine learning model gave a particular score, in terms of which input features mattered most.
Step-Up Challenge	A verification prompt triggered only for suspicious transactions, asking the user to prove their identity through biometrics or behaviour.
WebSocket	A persistent two-way connection between a server and a browser. Unlike normal web requests, the server can push data to the browser at any time without the browser asking.
XGBoost	Extreme Gradient Boosting — a highly effective machine learning algorithm that builds many small decision trees and combines them for accurate fraud classification.

10. Final Summary — The Three Transformations

Fin-Guardian AI represents three fundamental transformations in how financial fraud is handled:

Before Reactive — discover fraud after the money is gone	After Proactive — prevent fraud while the criminal is still on screen
Before Binary — block everything that looks unusual	After Adaptive — challenge only when suspicious, never destroy legitimate experience

Before

Isolated — each transaction evaluated alone

After

Networked — every transaction understood in the context of the entire financial graph

Fin-Guardian AI does not just detect fraud. It understands it.

And it does so in the time it takes you to blink.